

DNS & CDN at Scale

Why this chapter matters: before a request reaches your load balancer or servers, two systems have already shaped it: **DNS** decided *where* the client is sent, and a **CDN** may have answered it without touching your servers at all. They are the cheapest, highest-leverage scaling layers: they cut latency (distance is the one thing you cannot buy back, Chapter 3) and absorb most of the traffic. Chapter 3 explained how DNS resolution works; here we use DNS and CDNs as **design tools**.

↪ QUICK RECALL — Chapter map

- 6.1 The big picture → 6.2 DNS at scale → 6.3 DNS traffic-routing policies → 6.4 Anycast → 6.5 DNS failures & security
- 6.6 How a CDN works → 6.7 Push vs pull → 6.8 Cache keys & hit ratio → 6.9 Freshness & invalidation
- 6.10 Dynamic content, edge compute & security → 6.11 Video & large files → 6.12 Worked examples
- 6.13 Reference architecture & checklist → 6.14 Practice, Q&A, cheat sheet

6.1 The big picture: get closer, and cache

Both technologies apply the two ideas that have appeared in every chapter so far: **move data closer to the user** and **remember answers instead of recomputing them**.

	DNS	CDN
Decides	Which address the client connects to	Who serves the content , and whether the origin is contacted at all
Cached by	Browser, OS, recursive resolvers (TTL)	Edge servers worldwide (TTL, Cache-Control)
Best at	Steering users to the right region/server; failover	Offloading static & cacheable traffic; cutting latency and bandwidth
Typical failure	Stale answers, outage of the DNS provider	Stale content, cache misses stampeding the origin

◆ SENIOR INSIGHT — How they combine

- A typical production request: browser asks DNS for `www.example.com` → DNS (geo/latency policy or anycast) returns the **nearest CDN edge** → edge answers from cache (HIT) or fetches from a regional cache/origin (MISS) → origin is a load balancer plus app servers in one or more regions.
- Result: most users get a response in a few milliseconds from a few hundred kilometres away, and your origin sees only a small fraction of the traffic.

6.2 DNS at scale

Recap (Chapter 3): client → recursive resolver → root → TLD → **authoritative** name server → answer, cached for the record's TTL. At scale, what matters is how this hierarchy behaves under load and how you can **use it to your advantage**.

- **Why DNS scales:** aggressive caching at every level. Billions of lookups per day are answered by caches; authoritative servers see only misses. DNS is hierarchical, with a few authoritative servers at the top.
- **Your authoritative DNS** is usually a managed service (Amazon Route 53, Cloudflare, Google Cloud DNS, Azure DNS, NS1). They run thousands of anycast servers, give you APIs, health checks and traffic policies, and publish your NS records at the registrar.
- **Negative caching:** “this name does not exist” (NXDOMAIN) is also cached, for the time given in the zone's SOA record. Creating a record right after a failed lookup can still look broken for minutes.

- **Typical cost:** an uncached lookup costs a round trip or two to the resolver and more to authoritative servers (tens of milliseconds); cached lookups are almost free. It is the first of the ~4 round trips of a cold HTTPS request.
- **Lower-level resolvers cache mappings that can become stale** due to DNS propagation delays; browsers and operating systems add their own caches governed by the TTL.

TTL strategy

Situation	TTL choice	Reasoning
Stable records (MX, NS, rarely-changing sites)	Long: 1–24 hours	Fewer lookups, faster, resilient if DNS provider is briefly down
Records you may need to fail over or re-point (web/API entry points)	Short: 30–300 s	Faster failover and cutovers (blue-green) at the price of more queries
Planned migration	Lower the TTL a day or two before , switch, then raise it again	Caches expire in time for the switch; no mass-staleness
Records behind a health-checked load balancer	Alias/CNAME to the LB (managed DNS keeps IPs current)	The LB handles instance health; DNS rarely changes

✗ COMMON MISTAKE — TTL is a request, not a guarantee

- Some resolvers enforce minimum TTLs, some clients ignore TTL, and long-lived connections never re-resolve at all. Runtimes and libraries may cache DNS (older JVM settings can cache forever; Node keep-alive agents and connection pools keep using the old IP until the socket closes).
- So **do not rely on DNS alone for instant failover**: use health-checked load balancers, and make clients re-resolve periodically.

6.3 DNS traffic-routing policies (managed DNS as a traffic director)

Managed DNS can return **different answers to different users** or change answers when a health check fails. Services such as Cloudflare and Route 53 can route traffic through these methods:

Policy	How the answer is chosen	Use cases	Watch out
Simple / multi-value	Return one record, or several (up to ~8 healthy) and let the client pick (round robin)	Basic distribution across a few servers	Clients often try the first IP; no load awareness
Weighted round robin	Return records in proportion to weights (e.g., 90% / 10%)	Prevent traffic going to servers under maintenance (weight 0); balance between varying cluster sizes ; A/B testing and canary releases	Probabilistic, per-resolver (not per-user); cached answers skew the split
Latency-based	Route to the region with the lowest measured latency from the user's network	Multi-region apps; best user experience	Based on provider measurements, not your real server load
Geolocation	Route by the user's country/continent/state	Data residency & compliance (EU users → EU), localised sites, content licensing	Needs a default record; resolver location may differ from the user's
Geoproximity	Route to the geographically closest resource, with an adjustable bias	Shift load toward bigger regions	More tuning
Failover	Primary record until its health check fails, then the secondary	Active-passive disaster recovery; static "sorry" page on S3 as a last resort	Detection time + TTL = outage window (see 6.12)

Health checks (HTTP/TCP probes from multiple vantage points) remove unhealthy endpoints from answers automatically — the foundation of DNS failover. Typical rule: probe every 10–30 s, mark unhealthy after 2–3 consecutive failures.

◆ **SENIOR INSIGHT** — Limits of DNS-based load balancing

- **Round robin is distribution, not balancing:** DNS knows nothing about server load, connection counts or request cost, and a single caching resolver can send thousands of users to one IP.
- **Resolver location ≠ user location:** geo/latency policies see the recursive resolver's IP (e.g., a public resolver far away). **EDNS Client Subnet (ECS)** passes a truncated client prefix to fix this, with a privacy cost.
- **Caching delays changes:** removing a dead server only takes effect as caches expire. Therefore: DNS steers traffic **between regions or clusters**; a **load balancer (Chapter 7)** balances **within** a cluster with real health checks.
- Pattern — **GSLB (global server load balancing)**: geo/latency DNS → regional load balancer → availability-zone targets.

6.4 Anycast — one IP, many locations

With normal (**unicast**) addressing, one IP identifies one machine. With **anycast**, the **same IP is announced from many locations** via BGP, and the Internet's routing sends each packet to the **topologically nearest** location.

same IP 198.51.100.10 announced (BGP) from every PoP

User in Mumbai ----> [PoP Mumbai] User in Frankfurt ----> [PoP Frankfurt]
User in Tokyo ----> [PoP Tokyo] User in Sao Paulo ----> [PoP Sao Paulo]

If one PoP fails or is withdrawn, routing converges and traffic flows to the next nearest.

Benefits	Caveats
Automatic nearest-site routing with no DNS tricks Built-in failover (withdraw the route) DDoS absorption: attack traffic is spread over many sites Used by DNS root servers, public resolvers (1.1.1.1, 8.8.8.8), authoritative DNS providers and CDNs	"Nearest" is by BGP path, not always lowest latency Routing changes can move a user mid-connection → long-lived TCP connections may break (CDNs engineer around this) Harder to debug; needs BGP expertise or a provider

★ **MUST REMEMBER**

- CDNs steer users either with **DNS-based routing** (the CDN's name server returns the IP of a nearby edge) or with **anycast** (one IP everywhere) — often both.

6.5 DNS failure modes & security

Problem	What happens	Mitigation
DNS provider outage / DDoS	Your domain stops resolving, so everything looks down even though servers are healthy. A 2016 DDoS against a major DNS provider (Dyn) made many popular sites unreachable for hours	Use 2+ independent DNS providers (NS records for both), anycast providers, sensible TTLs so caches ride out short outages
DDoS on / through DNS (amplification)	Attackers spoof a victim's IP and send small queries that trigger large responses toward the victim	Rate-limit responses, close open resolvers, use a provider with DDoS protection
Cache poisoning / spoofing	A forged answer is accepted by a resolver, redirecting users to an attacker	DNSSEC (cryptographically signs records to prove authenticity and integrity — it does <i>not</i> encrypt), randomised ports/IDs, TLS everywhere (certificate check catches wrong sites)
Privacy	Plain DNS (UDP/53) reveals every domain a user visits	DoH / DoT (DNS over HTTPS / TLS)
Dangling records (subdomain takeover)	A CNAME points to a deleted cloud resource; an attacker claims it and serves content on your subdomain	Remove DNS records when decommissioning; scan for dangling CNAMEs
Mis-configuration	Wrong TTL, missing record, expired domain, CNAME at the apex, broken delegation	Infrastructure-as-code for DNS, change review, monitoring of resolution from multiple regions, domain auto-renew + registry lock

Disadvantages of DNS (summary): accessing a DNS server adds a slight delay (mitigated by caching); server management can be complex and is generally run by governments, ISPs and large companies; DNS services can be DDoS targets, preventing users from reaching sites such as Twitter without knowing the IP addresses.

◆ SENIOR INSIGHT — DNS is the dependency of every dependency

- Inside a cluster, DNS is also the **service-discovery** layer (Kubernetes CoreDNS, Consul DNS). A slow or overloaded internal DNS makes every service call slow — and in Node, `dns.lookup` runs on the `libuv` thread pool of 4 threads (Chapter 1), so a DNS stall can starve unrelated file and crypto operations. Cache lookups in-process, use a local caching resolver, and monitor DNS latency as a first-class metric.

6.6 How a CDN works

A **content delivery network** is a **globally distributed network of proxy (cache) servers** that serve content from locations closer to the user. Generally static files — HTML/CSS/JS, photos, videos, downloads — are served from the CDN, although many CDNs (e.g., Amazon CloudFront, Cloudflare, Fastly, Akamai) also accelerate dynamic content. The site's DNS resolution tells clients which server to contact.

Vocabulary

Term	Meaning
Origin	Your own servers or object storage (e.g., an S3 bucket) holding the source of truth
Edge / PoP (Point of Presence)	A CDN data centre close to users (hundreds worldwide) with caching servers
Regional / mid-tier cache, origin shield	An extra cache layer between edges and the origin so that many edge misses collapse into one origin request
Cache HIT / MISS	HIT = served from the edge cache; MISS = edge must fetch from the next layer
TTL	How long an object may be served from cache before revalidation or refetch

Request flow

```
graph TD
    User -- DNS -->|cdn.example.com (CNAME to the CDN; CDN's DNS/anycast returns the nearest edge)| EdgePoP[Edge PoP near user]
    EdgePoP -- HIT -->|respond ms| Respond1[respond ms]
    EdgePoP -- MISS --> RegionalCache[Regional cache / origin shield]
    RegionalCache -- HIT -->|edge stores copy --> respond| Respond2[respond]
    RegionalCache -- MISS --> Origin[Origin: load balancer + app, or object storage]
    Origin -->|shield & edge store per TTL --> respond| Respond3[respond]
```

Serving content from a CDN improves performance in two ways:

- 1 **Users receive content from data centres close to them** — fewer kilometres, fewer hops, lower RTT, and TCP/TLS handshakes terminate at a nearby edge.
- 2 **Your servers do not have to serve the requests the CDN fulfils** — origin load, bandwidth cost and the risk of overload fall sharply.

★ MUST REMEMBER — More benefits you can mention in interviews

- **Availability:** can serve stale cached content if the origin is down (stale-if-error). **Security:** DDoS absorption across the global edge, WAF, bot filtering, TLS termination. **Cost:** egress from a CDN is usually cheaper than from your cloud region. **Scalability:** a traffic spike (viral post, product launch) is absorbed at the edge.

🔗 CORRECTION TO YOUR ORIGINAL NOTE — “CDNs only serve static content”

- That was true historically; your notes already point out that some CDNs (e.g., CloudFront) support dynamic content. Today CDNs also provide **dynamic site acceleration, API caching, edge compute and streaming** (6.10–6.11).

6.7 Push CDNs vs pull CDNs

	Push CDN	Pull CDN
How content gets to the CDN	You upload content to the CDN whenever it changes. You take full responsibility for providing content, uploading directly to the CDN and rewriting URLs to point to the CDN	CDN pulls from your origin when the first user requests it. You leave content on your server and rewrite URLs to point to the CDN
Control	You configure when content expires and when it is updated	TTL (from Cache-Control headers or CDN rules) determines how long content is cached
Traffic & storage	Content is uploaded only when new or changed → minimises traffic, maximises storage (everything is stored whether requested or not)	Minimises storage on the CDN (only requested objects stay), but can create redundant traffic if files expire and are re-pulled before they actually changed
First request	Fast (already there)	Slower until the content is cached on the CDN (cache-miss penalty)
Operational effort	Higher: you manage uploads/versioning/deletes	Low: just point the CDN at the origin
Best for	Sites with a small amount of traffic or content that isn't often updated — content is placed on the CDN once instead of being re-pulled at regular intervals	Sites with heavy traffic — load is spread more evenly, only recently requested content stays on the CDN

◆ SENIOR INSIGHT — In practice: pull is the default, with these refinements

- **Revalidation** removes the “redundant traffic” problem: when a cached object expires, the edge sends If-None-Match/If-Modified-Since; the origin answers **304 Not Modified** (no body) if it hasn’t changed (Chapter 4).
- **Pre-warming / preloading** of known-hot objects (a launch video, a new release) turns pull into push-like behaviour. Upload to **object storage as origin** (e.g., S3 + CDN) and you get ‘push to origin, pull to edge’ — the most common modern setup.
- Disadvantages of CDNs: cost can be significant depending on traffic (weigh it against what you would pay serving everything yourself); content may be stale if updated before the TTL expires; you must change URLs for static content to point to the CDN.

6.8 Cache keys, headers & hit ratio

The **cache key** decides whether two requests are ‘the same object’. By default it is the host + path + query string; CDNs let you add or remove parts (query parameters, cookies, headers such as Accept-Encoding).

Pitfall	Effect on hit ratio	Fix
Unique query strings (?utm_source=..., timestamps, session ids)	Every URL looks different → cache fragmentation , near-0% hits	Ignore irrelevant parameters in the cache key; only whitelist the ones that change the response
Vary: Cookie or Vary: User-Agent	One copy per cookie / per browser string → explosion of variants	Vary only on what matters (Accept-Encoding); normalise device classes (mobile/desktop)
Set-Cookie on static responses; Cache-Control: private/no-store	CDN refuses to cache	Serve static assets from a cookieless domain/path; mark them public
Short or missing TTL on assets	Constant revalidation, origin load	Long TTL + fingerprinted filenames (6.9)
Low-traffic long-tail objects	Rarely requested → expire before the next hit; each PoP caches separately	Origin shield / tiered caching so one PoP’s miss fills the others

Useful headers for CDNs (Chapter 4 recap): Cache-Control: max-age=... (browser and shared caches), s-maxage=... (**shared caches only**, i.e., the CDN), public/private, stale-while-revalidate, stale-if-error, ETag, and CDN-specific headers (Surrogate-Key, CDN-Cache-Control).

Measuring effectiveness

Metric	Definition	Why it matters
Request hit ratio	hits ÷ total requests	Shows how many requests the origin is spared
Byte hit ratio	bytes served from cache ÷ total bytes	Matters for bandwidth cost — a few huge video files can dominate
Origin offload	origin requests (or bytes) ÷ total	Origin traffic = (1 – hit ratio) × total traffic
TTFB at the edge / p95 latency	time to first byte by region	Verifies the latency benefit
Origin shield hit ratio, error rate (4xx/5xx), purge latency	—	Detects mis-keyed caches and origin trouble

★ MUST REMEMBER

- Good targets: **> 90-95% for static assets**. Going from 90% to 99% cuts origin traffic **10x** (10% → 1%) — always think in **miss rate**, as in Chapter 2.

6.9 Freshness & cache invalidation

Content may be **stale** if it is updated before the TTL expires it. Deciding how and when cached copies are replaced is the classically hard part of caching.

Strategy	How it works	Pros	Cons
TTL expiry	Copy simply expires after N seconds/hours	Zero effort, always eventually fresh	Updates are delayed up to the TTL
Versioned / fingerprinted URLs	Filename contains a content hash: <code>app.4f9c2.js</code> . Change content → new URL. Cache it for a year with <code>immutable</code>	Best practice for assets: instant 'invalidation', perfect caching, no purge needed	HTML that references the files must itself stay fresh; build tooling required
Purge / invalidate	API call removes objects by URL, wildcard/prefix, or tag (surrogate key) ; 'purge all' as a last resort	Immediate-ish control for HTML, API responses, takedowns	Propagates in seconds to minutes; providers may limit or charge; purge-all causes a miss storm
Stale-while-revalidate / stale-if-error	Serve the stale copy instantly while refreshing in the background, or when the origin errors	Low latency + resilience	Users may briefly see old data
Short TTL + revalidation	TTL of seconds to minutes; edge revalidates with ETag	Near-fresh with low origin load	More origin hits than long TTL

★ MUST REMEMBER — The standard recipe

- **HTML / API responses:** short TTL or no-cache with ETag (or purge on publish). **Static assets (JS, CSS, images, fonts):** fingerprinted filenames + `Cache-Control: public, max-age=31536000, immutable`. **Deploy** = upload new assets first, then update the HTML that points to them.

◆ SENIOR INSIGHT — Cache stampede (thundering herd) at the edge

- When a very popular object expires or is purged, thousands of simultaneous requests all miss and hit the origin together. Defences: **request collapsing/coalescing** (the edge sends one origin request and makes the others wait), **origin shield**, **stale-while-revalidate**, **staggered/jittered TTLs**, and pre-warming before a planned purge.
- Purging a hot key is a deliberate, risky operation — prefer fingerprinted URLs and tag-based purge of small sets over 'purge everything'.

6.10 Dynamic content, edge compute & security at the edge

Capability	What it does	Example
Dynamic site acceleration	Uncacheable requests still pass through the edge: the user's TCP/TLS handshakes end at a nearby edge , and the edge reuses warm, long-lived, optimised connections to the origin	Cuts round trips: user→edge 15 ms instead of user→origin 200 ms per handshake step
Micro-caching	Cache dynamic GET responses for 1–5 seconds	A trending API endpoint hit 20,000x/s becomes ~1 origin request/s per PoP; users see ≤ 5 s old data
Edge compute (Cloudflare Workers, Lambda@Edge/CloudFront Functions, Fastly Compute)	Run small code at the edge before/after the cache	Redirects, A/B bucketing, auth token checks, geo-personalisation, image resizing, header rewriting
Image / media optimisation	Resize, convert (WebP/AVIF), compress on the fly per device	?w=400&fmt=webp variants cached separately
Security services	DDoS mitigation (L3/L4/L7), WAF , bot management, rate limiting, TLS certificates at the edge	Attack traffic is dropped at the edge, never reaching your origin
Access control	Signed URLs / signed cookies with expiry for private or paid content; hotlink protection; geo-blocking	A video link valid for 10 minutes for one user

✗ COMMON MISTAKE — Origin bypass

- If attackers learn your origin's IP, they can skip the CDN and its WAF/DDoS shield. **Lock the origin down:** allow only the CDN's IP ranges (or a private link / mTLS) and require a secret header; use TLS between edge and origin; don't expose the origin hostname in DNS history or certificates.

◆ SENIOR INSIGHT — Privacy and consistency traps

- Never cache responses that contain **personal data** under a shared key (the classic leak: one user's 'my account' page served to another). Cache by default *only* what is explicitly safe; mark personalised responses private, no-store and keep cookies/Authorization out of cacheable paths.
- Edge logic must be fast and stateless; global state at the edge is eventually consistent (edge key-value stores replicate asynchronously — Chapter 5).

6.11 Video & large-file delivery

- **Segmented streaming** (HLS, MPEG-DASH): a video is cut into 2–10 second segments at several bitrates; the player downloads segments over plain HTTP and switches bitrate to match bandwidth (**adaptive bitrate**). Segments are ordinary static files, so CDNs cache them perfectly. Live streams use the same idea with a rolling playlist.
- **Range requests** (Range: bytes=...) let players seek and let CDNs cache large files in chunks; downloads can resume.
- **Going deeper than the edge:** large streamers place cache appliances **inside ISP networks** (for example Netflix's Open Connect) and pre-fill them during off-peak hours with the titles predicted to be popular, so most traffic never crosses the wider Internet.
- **Multi-CDN:** use two or more CDNs for resilience and price/performance, with DNS or client-side steering based on real-user measurements and health. It adds complexity (config parity, cache fill, certificates) — worth it for very large or critical services.

- **Upload path:** large uploads usually go directly to object storage with pre-signed URLs (bypassing your servers), then asynchronous transcoding workers (queues, Chapter 12) produce renditions that the CDN serves.

6.12 Worked examples

Example A — CDN offload (continuing Chapter 2's photo app: ≈ 9.3 Gbps average egress for images, 95% CDN hit ratio)

Step	Calculation	Result
Total image egress	$5,800 \text{ req/s} \times 200 \text{ KB} \approx 1.16 \text{ GB/s}$	$\approx 9.3 \text{ Gbps}$ average (peak 2-3 $\times \approx 20$ -28 Gbps)
Origin traffic after CDN	$(1 - 0.95) \times 9.3 \text{ Gbps}$	$\approx 0.47 \text{ Gbps}$ ($\sim 58 \text{ MB/s}$)
If hit ratio were 80%	$0.20 \times 9.3 \text{ Gbps}$	$\approx 1.9 \text{ Gbps}$ — 4 $\times$ more origin load $\rightarrow$ fix cache keys/TTL
Requests reaching origin	$5,800 \times 0.05$	$\approx 290 \text{ req/s}$ — comfortably served by a few object-store/app nodes

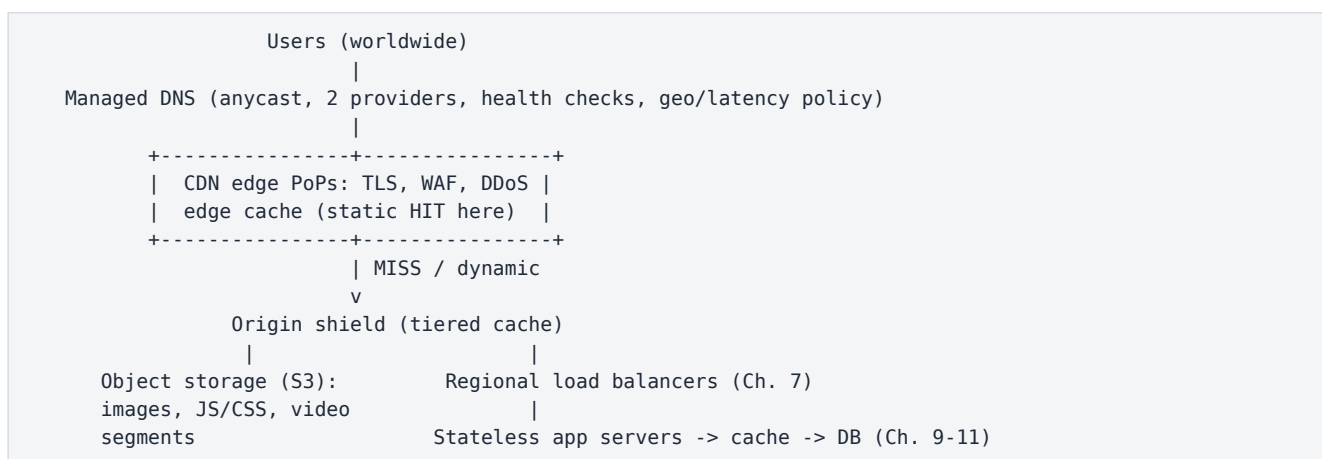
Example B — Latency from distance (user in Mumbai, origin in Virginia ≈ 200 ms RTT; edge in Mumbai ≈ 15 ms RTT)

	Origin only	Via CDN edge (cache HIT)
DNS + TCP + TLS 1.3 + HTTP (≈ 4 RTT, Chapter 3)	$4 \times 200 \text{ ms} = 800 \text{ ms}$ to first byte	$4 \times 15 \text{ ms} = 60 \text{ ms}$
Improvement	—	$\approx 13\times$ faster; on a MISS the edge reuses a warm connection to the origin, so the user still pays only the first leg's handshakes (~ 60 ms) plus one origin round trip

Example C — DNS failover time (health check every 30 s, unhealthy after 3 failures, record TTL 300 s)

detection = 3 failures $\times$ 30 s = 90 s
 cache wait = up to the record TTL = up to 300 s
 worst-case = 90 s + 300 s ~ 6.5 minutes (average ~ 4 minutes)
 With TTL 60 s and 10 s checks (3 failures): 30 + 60 = ~ 90 s worst case. Resolvers/clients that ignore TTL or hold long-lived connections can stretch this - so keep a load balancer for fast failover.

6.13 Reference architecture & design checklist



Checklist

- 1 Put **all static assets** (and user media) behind a CDN with an object-storage origin.
- 2 Use **fingerprinted filenames** + long TTL for assets; short TTL / revalidation for HTML and APIs.
- 3 Design **cache keys** deliberately; strip tracking params; keep cookies off cacheable paths.
- 4 Enable **origin shield** / tiered caching, request collapsing and **stale-if-error**.
- 5 **Lock down the origin** to the CDN; enable WAF, rate limiting and DDoS protection.
- 6 Use **managed, multi-provider DNS** with health checks; choose TTLs per record; plan migrations by lowering TTL first.
- 7 Use DNS geo/latency routing only to choose a **region**; use load balancers inside it.
- 8 Monitor **hit ratio (request and byte)**, origin offload, edge error rates, DNS resolution times and certificate expiry.
- 9 Test failure: a CDN or DNS-provider outage, a purge-all, a cold cache after launch.

6.14 Practice questions (try before looking)

P1. A site serves 4 Gbps of static traffic at a 92% CDN hit ratio. How much reaches the origin? What if it improves to 98%?

Answer: $8\% \times 4 = 0.32 \text{ Gbps}$; $2\% \times 4 = 0.08 \text{ Gbps}$ — a 4x reduction from six points of hit ratio.

P2. Your DNS TTL is 1 hour and the primary region dies. Is DNS failover a good primary strategy for this?

Answer: No: clients/resolvers may keep the dead IP up to ~1 hour (plus detection time). Use a health-checked load balancer / anycast for fast failover and keep DNS TTL low for such records.

P3. You deploy a new JavaScript bundle but users still run the old one for hours. Why, and what is the permanent fix?

Answer: The old file is cached (browser/CDN) under the same URL until TTL expires. Fix: **fingerprinted filenames** (app.4f9c2.js) with immutable long TTL, and short-TTL HTML that references them.

P4. Which CDN model — push or pull — for (a) a small marketing site updated twice a year, (b) a news site with millions of daily visitors?

Answer: (a) Push (or pull with long TTL) — upload once, tiny traffic. (b) **Pull** with origin shield — only requested content is cached, load spreads automatically.

P5. Hit ratio is only 20% although assets are static. Name three likely causes.

Answer: Unique query strings in the cache key; Vary: Cookie/User-Agent; Set-Cookie/private/no-store headers on assets; TTLs too short.

Interview Q&A (30-second answers)

Q1. What is a CDN and why use one?

A globally distributed network of caching proxies that serves content from near the user. It cuts latency, offloads the origin, absorbs spikes and attacks, and can serve stale content during origin outages.

Q2. Push vs pull CDN?

Push: you upload content (control, minimal traffic, more storage, best for small/rarely-updated sites). Pull: CDN fetches on first request (low effort, minimal storage, first-request penalty, best for heavy traffic).

Q3. How do you handle cache invalidation with a CDN?

Prefer fingerprinted URLs with long TTLs (no invalidation needed); for HTML/APIs use short TTLs + ETag revalidation or targeted purge by URL/tag; protect against stampedes with collapsing and stale-while-revalidate.

Q4. How does a user get routed to the nearest edge?

Either DNS-based (the CDN's DNS answers with a nearby edge IP using geo/latency data, ideally with EDNS Client Subnet) or anycast (same IP announced from every PoP; BGP picks the nearest).

Q5. What are DNS routing policies?

Simple/multi-value, weighted (maintenance, cluster sizes, A/B tests), latency-based, geolocation (compliance), geoproximity, and health-checked failover.

Q6. Why isn't DNS round robin real load balancing?

It is blind to load and health, answers are cached per resolver so distribution is lumpy, and removing a dead server waits for TTL expiry. Use LBs inside a region.

Q7. What is anycast?

Announcing the same IP from many locations so routing delivers each client to the nearest one; gives low latency, failover and DDoS dilution. Used by DNS roots and CDNs.

Q8. How do you reduce DNS as a single point of failure?

Multiple independent authoritative providers, anycast, reasonable TTLs, monitoring from several regions, and not relying on DNS alone for failover.

Q9. What can go wrong when caching dynamic content at a CDN?

Serving one user's personalised response to another (bad cache key / missing private), stale data after updates, and Vary explosions. Cache only what's safe; micro-cache with short TTLs.

Q10. How would you serve video globally?

Transcode to adaptive-bitrate HLS/DASH segments in object storage, serve through a CDN (possibly multi-CDN), signed URLs for access control, with edge caches pre-filled for popular titles.

Chapter 6 Cheat Sheet

◀ QUICK RECALL — Revise this in 2 minutes

- **DNS decides where; CDN decides who serves.** Both are caches that put data closer to users.
- **DNS TTL:** long = fewer lookups, resilient; short = fast failover. Lower TTL **before** migrations. TTL is advisory — don't rely on DNS alone for failover.
- **Routing policies:** simple/multivalue • weighted (maintenance, cluster sizes, A/B) • latency • geolocation (compliance) • geoproximity • failover (health checks).
- **DNS ≠ load balancer:** no load awareness, cached answers. Pattern = geo DNS → regional LB → AZ. Resolver location ≠ user location (ECS).
- **Anycast:** same IP everywhere, BGP picks nearest; used by roots, resolvers, CDNs; DDoS absorption.
- **DNS risks:** provider outage/DDoS (use 2+ providers), cache poisoning (DNSSEC signs, doesn't encrypt), amplification, dangling CNAMEs, privacy (DoH/DoT).
- **CDN flow:** user → nearest edge → HIT or MISS → regional cache/shield → origin. Benefits: proximity + origin offload (+ security, availability).
- **Push** = you upload (small/rarely changing). **Pull** = fetch on first request (heavy traffic). Modern default: S3 origin + pull CDN + shield.
- **Hit ratio:** origin traffic = $(1 - \text{hit}) \times \text{total}$. Fix cache keys: strip query noise, limit Vary, no cookies on assets.
- **Freshness:** fingerprinted filenames + 1-year immutable for assets; short TTL/ETag or purge for HTML/API; stale-while-revalidate; guard against stampedes.
- **Edge extras:** dynamic acceleration, micro-caching, edge compute, WAF/DDoS, signed URLs. Lock the origin to the CDN.
- **Numbers:** cold HTTPS ≈ 4 RTT: 800 ms at 200 ms RTT vs 60 ms at a 15 ms edge. DNS failover $\approx$ detection + TTL.